Programowanie obiektowe i inżynieria oprogramowania

Lab 02:

- wstęp do programowania obiektowego;
- wstęp do projektu kubeczków

Akademia Górniczo-Hutnicza im. Stanisława Staszica w Krakowie
AGH University of Science and Technology

Prowadzący: dr inż. Jakub Bryła
2024

Mini projekt *Kubeczki*

- Projekt stanowi wprowadzenie do programowania obiektowego w C++.

- Zadaniem programu będzie odwzorowanie mieszania się składników w i pomiędzy kubeczkami.

Pliki nagłówkowe

Zaczynamy od plików nagłówkowych, w których opisujemy szkielet naszych klas, tworzymy ich zarys bez wchodzenia w szczegóły (pisania kodu funkcji).

TCup.cpp | TCup.h | TSubstance.cpp | TSubstance.h

kubki | (Globalny zasięg)

```
1  #pragma once
2  #include <iostream>
3
4
5  class TSubstance
6  {
7      std::string name;
8      double      ro;       // [kg/m3]
9
10 public:
11     TSubstance(std::string name, double ro);
12
13     std::string get_name();
14     double      get_ro();
15 };
16
```

Tworzymy klasę opisującą właściwości pojedynczej substancji

TCup.cpp | TCup.h | TSubstance.cpp | TSubstance.h

kubki | TCup

```
1  #pragma once
2  #include <iostream>
3  #include <vector>
4
5  #include "TSubstance.h"
6
7  class TCup
8  {
9      std::vector<TSubstance> substances;
10     std::vector<double> volumes;
11
12 public:
13     void add(TSubstance substance, double volume_in_ml);
14     void show();
15 };
16
17
```

Tworzymy klasę opisującą kubek

Pliki nagłówkowe

Obiekty utworzone na bazie klasy charakteryzują się:

- cechami / właściwościami, przechowywanymi w postaci zmiennych klasy,
- funkcjonalnościami zaimplementowanymi w postaci funkcji klasy.

TCup.cpp | TCup.h | TSubstance.cpp | TSubstance.h

kubki | (Globalny zasięg)

```
1  #pragma once
2  #include <iostream>
3
4
5  class TSubstance
6  {
7      std::string name;
8      double      ro;       // [kg/m3]
9
10 public:
11     TSubstance(std::string name, double ro);
12
13     std::string get_name();
14     double      get_ro();
15 };
16
```

Substancje rozróżniamy za pomocą ich nazwy (woda, soczek itd.). Każda substancja charakteryzuje się pewną gęstością. **Nazwa i gęstość są cechami obiektu** (utworzonego na bazie przepisu – klasy).

TCup.cpp | TCup.h | TSubstance.cpp | TSubstance.h

kubki | TCup

```
1  #pragma once
2  #include <iostream>
3  #include <vector>
4
5  #include "TSubstance.h"
6
7  class TCup
8  {
9      std::vector<TSubstance> substances;
10     std::vector<double> volumes;
11
12 public:
13     void add(TSubstance substance, double volume_in_ml);
14     void show();
15 };
16
17
```

Kubki będą różnić się zawartością – wlanymi substancjami i ich objętością. **Obie informacje (pojemność i rodzaj wlanych cieczy) musimy przechowywać w zmiennych obiektu.**

Kubek posiada niesamowitą **funkcjonalność** wlewania do niego kolejnych substancji. Deklarujemy funkcję *add()*.

Pliki nagłówkowe

TCup.cpp | TCup.h | TSubstance.cpp | TSubstance.h

kubki | (Globalny zasięg)

```
1   #pragma once
2   #include <iostream>
3
4
5   class TSubstance
6   {
7       std::string name;
8       double      ro;       // [kg/m3]
9
10  public:
11      TSubstance(std::string name, double ro);
12
13      std::string get_name();
14      double      get_ro();
15  };
16
```

Jedną z dobrych praktyk programowania obiektowego jest ograniczenie dostępu do atrybutów (zmiennych) klasy. Dostęp do zmiennych *name* i *ro* jest możliwy jedynie w zakresie klasy. **Podejście to nazywa się enkapsulacją/hermetyzacją**.

W celu odczytania wartości atrybutów obiektu tworzymy dwie funkcje/metody: *get_name()* i *get_ro()*.

TCup.cpp | TCup.h | TSubstance.cpp | TSubstance.h

kubki | TCup

```
1   #pragma once
2   #include <iostream>
3   #include <vector>
4
5   #include "TSubstance.h"
6
7   class TCup
8   {
9       std::vector<TSubstance> substances;
10      std::vector<double> volumes;
11
12  public:
13      void add(TSubstance substance, double volume_in_ml);
14      void show();
15  };
16
17
```

Głównie do celów **debugowania/testowania** poprawnego działania programu będziemy chcieli wyświetlać informacje o kubeczku. Informacje te będą wyświetlane w konsoli programu po wywołaniu metody *show()*.

Pliki nagłówkowe

TCup.cpp | TCup.h | TSubstance.cpp | TSubstance.h

kubki | (Globalny zasięg)

```
1   #pragma once
2   #include <iostream>
3
4
5   class TSubstance
6   {
7       std::string name;
8       double      ro;       // [kg/m3]
9
10  public:
11      TSubstance(std::string name, double ro);
12
13      std::string get_name();
14      double      get_ro();
15  };
16
```

Deklarujemy istnienie specjalnej funkcji, tzw. **konstruktora**. Jest to funkcja, która pozwoli nam opisać zadania do wykonania w trakcie tworzenia obiektu klasy.

- Konstruktor to funkcja, która ma tą samą nazwę co klasa – w tym przypadku TSubstance.
- Konstruktur nie zwraca żadnej wartości – po lewej stronie nie ma typu zwracanego.

Pliki źródłowe

TCup.cpp | TCup.h | TSubstance.cpp | TSubstance.h

kubki | (Globalny zasięg)

```
1   #include "TSubstance.h"
2   using namespace std;
3
4   TSubstance::TSubstance(string name, double ro)
5   {
6       this->name = name;
7       this->ro = ro;
8   }
9
10  string TSubstance::get_name()
11  {
12      return name;
13  }
14
15  double TSubstance::get_ro()
16  {
17      return ro;
18  }
19
```

W konstruktorze przypisujemy podane wartości (nazwę i gęstość cieczy) do atrybutów tworzonego obiektu (substancji).

Nazwy pomocnicze parametrów wejściowych funkcji (konstruktora), określone w linijce 4, są takie same jak nazwy atrybutów. Mamy do czynienia z *przysłanianiem nazwy zmiennej*. W celu rozróżnienia kiedy mamy na myśli parametr wejściowy, a kiedy atrybut stosuje się zapis *this->nazwa_zmiennej* dla atrybutów.

Funkcje *get_name()* i *get_ro()* mają za zadanie jedynie zwrócić wartość poszczególnych atrybutów obiektu.

Pliki źródłowe

kubki.cpp

kubki (Globalny zasięg)

```cpp
// kubki.cpp : Ten plik zawiera funkcję „main”. W
//

#include <iostream>
#include <vector>
#include "TSubstance.h"
#include "TCup.h"
using namespace std;

vector<TSubstance> substancje =
{
    TSubstance{"woda", 997.8},
    TSubstance{"mleko", 1030.0},
    TSubstance{"oliwa", 920.0},
    TSubstance{"krew", 1050.0}
};

int main()
{...}
```

Na tym etapie możemy utworzyć listę znanych substancji, które już niedługo będziemy mogli wlać do kubka.

Dla wygody skorzystamy z wektora, umożliwiającego łatwą modyfikację listy substancji.

Obiekt tworzymy na podstawie utworzonego przepisu – klasy.

Obiekt powołujemy do życia podając nazwę klasy i wartości parametrów wymaganych przez konstruktor:

- *TSubstance(„woda”, 997.8)*

lub

- *TSubstance{„woda”, 997.8}*

Substancje przechowywane są w wektorze pod wspólną nazwą „substancje”.

Pliki źródłowe

Sprawdźmy czy kod działa:

```
int main()
{
        for (int i=0; i<4; i++)
        {
                cout << subtancje[i].get_name() << „: „ << subtancje[i].get_ro() << „kg/m3\n”;
        }
}
```

Pliki źródłowe

TCup.cpp | TCup.h | TSubstance.cpp | TSubstance.h

kubki | (Globalny zasięg)

```
1   #include "TCup.h"
2   using namespace std;
3
4   void TCup::add(TSubstance substance, double volume_in_ml)
5   {
6       substances.push_back(substance);
7       // volumes.push_back(volume_in_ml / 1000 / 1000);
8       volumes.push_back(volume_in_ml / 1e6);
9   }
10
11  void TCup::show()
12  {
13      int count = substances.size();
14      for (int i = 0; i < count; i++)
15      {
16          double mass = substances[i].get_ro() * volumes[i] * 1000;       // grams
17          cout << substances[i].get_name()
18              << "; volume: " << volumes[i] * 1e6 << " ml"
19              << "; mass: " << mass << " g" << endl;
20      }
21  }
```

Linijka 16: wywołujemy funkcję obiektu klasy TSubstance wewnątrz klasy TCup.

Pliki źródłowe

kubki.cpp* — kubki — (Globalny zasięg)

```cpp
// kubki.cpp : Ten plik zawiera funk
//

#include <iostream>
#include <vector>
#include "TSubstance.h"
#include "TCup.h"
using namespace std;

vector<TSubstance> substancje =
{
    TSubstance{"woda", 997.8},
    TSubstance{"mleko", 1030.0},
    TSubstance{"oliwa", 920.0},
    TSubstance{"krew", 1050.0}
};

int main()
{
    cout << "Hello World!\n";

    TCup kubek1;
    kubek1.add(substancje[0], 100);
    kubek1.add(substancje[1], 50);
    kubek1.show();
}
```

Konsola debugowania programu Microsoft Visual Studio

```
Hello World!
woda; volume: 100 ml; mass: 99.78 g
mleko; volume: 50 ml; mass: 51.5 g
```

Obiekt tworzymy na podstawie utworzonego przepisu – klasy.

Obiekt powołujemy do życia podając nazwę klasy, nazwę obiektu i wartości parametrów wymaganych przez konstruktor (w przypadku kubka nie mamy konstruktora, kubek nie posiada atrybutów którym trzeba nadać wartości początkowe):

- *TCup kubek1;*

Funkcję/metodę obiektu wywołujemy podając:

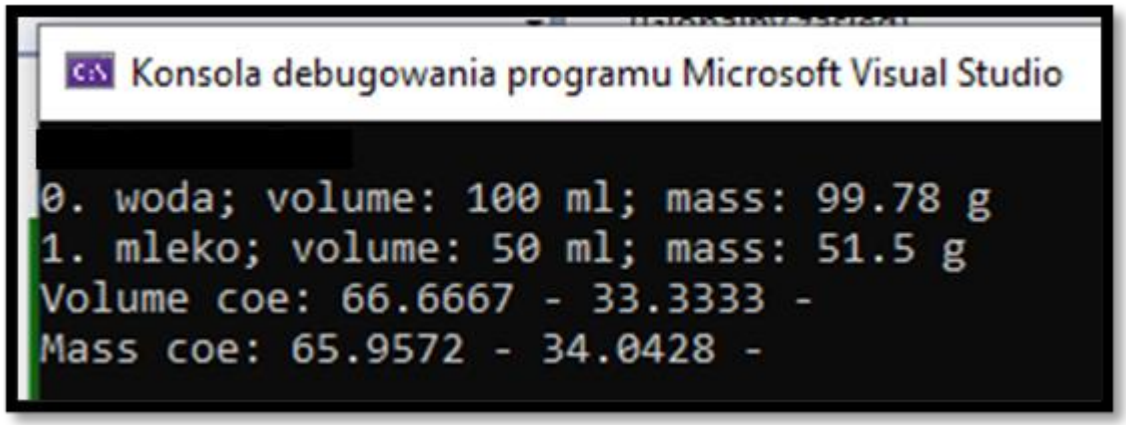
- nazwę obiektu (na rzecz którego wywołujemy funkcję),
- *kropkę „.”*
- *nazwę funkcji z jej argumentami w nawiasach okrągłych.*

kubek1.add(substancje[0], 100);
kubek1.show();

Zadanie

Zadanie polega na rozbudowaniu klasy *TCup* o wyświetlanie informacji o:

- udziale objętościowym poszczególnych substancji w kubku,
- udziale masowym poszczególnych substancji w kubku.



Zadanie

TSubstance.h | TCup.h

Kubeczki_zajecia1 | (Globalny zasięg)

```cpp
#pragma once
#include <vector>

#include "TSubstance.h"

class TCup
{
    std::vector<TSubstance> substances;
    std::vector<double> volumes;

public:
    void add(TSubstance substance, double volume_in_mm);
    void show();

private:
    void print_coe_vol();
    void print_coe_mass();
};

```

main.cpp | TSubstance.cpp | TCup.cpp

Kubeczki_zajecia1 | (Globalny zasięg)

```cpp
    {
        double mass = substances[i].get_ro() * volumes[i] * 1e3; // [g]

        cout << substances[i].get_name()
            << "; volume: " << volumes[i] * 1e6 << "ml"
            << "; mass: " << mass << "g" << endl;
    }

    // skorzystanie z funkcji get_coe_vol i get_coe_mass
    print_coe_vol();
    print_coe_mass();
    cout << endl;
}

void TCup::print_coe_vol()
{
    cout << "\nVolume coe: ";
    // implementacja
}

void TCup::print_coe_mass()
{
    cout << "\nMass coe: ";
    // implementacja
}

```